

Contents

Disjunktni skupovi (Union-Find)	1
1. Problem	1
Tipične primene	1
2. Jednostavna (naivna) struktura podataka	1
3. Efikasna struktura: šuma (forest) sa path compression i union by rank/size	2
3.1 Union by rank (ili union by size)	2
3.2 Path compression (kompresija putanje)	3
3.3 Vremenska složenost	3
3.4 Poređenje	3
4. Implementacija u C++ (kompletan primer)	3
5. Primeri primene	4
5.1 Detekcija ciklusa / povezane komponente	4
5.2 Kruskalov algoritam (MST)	4
5.3 "Prijateljski krugovi" / equivalence klase	5
6. Zaključak	5

Disjunktni skupovi (Union-Find)

1. Problem

Disjunktni skupovi (engl. *disjoint sets*) predstavljaju kolekciju skupova $S_1, S_2, \dots, S_k$ takvih da:

- svaki element pripada tačno jednom skupu ($S_i \cap S_j = \emptyset$ za $i \neq j$),
- unija svih skupova čini ceo univerzum elemenata.

Nad ovakvom strukturom najčešće su potrebne tri operacije:

1. **MakeSet(x)** – kreira novi skup koji sadrži samo element x.
2. **Find(x)** – vraća identifikator (predstavnik) skupa kojem element x trenutno pripada. Koristi se da bismo proverili da li dva elementa pripadaju istom skupu.
3. **Union(x, y)** – spaja skupove kojima pripadaju x i y u jedan skup.

Ova apstraktna struktura naziva se **Union-Find** ili **Disjoint Set Union (DSU)**.

Tipične primene

- Kruskalov algoritam za minimalno razapinjuće stablo (provera da li dodavanje grane pravi ciklus).
- Određivanje povezanih komponenti grafa (offline, bez BFS/DFS).
- Detekcija ciklusa u neusmerenom grafu.
- Perkolacija, klasterizacija, dinamička povezanost mreže.
- Rešavanje problema tipa "ko je sa kim u istom timu/grupi" (equivalence relations).

2. Jednostavna (naivna) struktura podataka

Najprirodnije rešenje je da svaki element ima **oznaku (labelu)** skupa kojem pripada, npr. niz $id[1..n]$, gde $id[x]$ govori kom skupu element x pripada.

MakeSet(x): $id[x] = x$ // 0(1)

Find(x): return $id[x]$ // 0(1)

```

Union(x, y):
    a = Find(x)
    b = Find(y)
    if a == b: return
    for i = 1 to n:                // O(n)
        if id[i] == a:
            id[i] = b

```

Analiza: - Find je $O(1)$ - trivijalno brzo. - Union je $O(n)$ jer moramo proći kroz sve elemente i preimenovati oznaku starog skupa u novi.

Ako izvršimo niz od m operacija Union nad n elemenata, u najgorem slučaju dobijamo **$O(n \cdot m)$** vremena - potpuno neefikasno za velike skupove podataka (npr. $n = 10^6$, $m = 10^6 \rightarrow 10^{12}$ operacija).

Problem je što je Union "skup" operacija koja mora da ažurira potencijalno sve elemente, dok bi bilo bolje da rad prebacimo na Find, koji se poziva mnogo ređe u praksi ili se lakše ubrzava.

3. Efikasna struktura: šuma (forest) sa path compression i union by rank/size

Umesto niza labela, koristimo **implicitnu šumu stabala**, gde svaki element ima pokazivač na svog roditelja: `parent[x]`. Predstavnik skupa je **koren** stabla (element čiji je `parent[x] == x`).

```

MakeSet(x):
    parent[x] = x
    rank[x] = 0           // (ili size[x] = 1)

```

```

Find(x):
    while parent[x] != x:
        x = parent[x]
    return x

```

```

Union(x, y):
    rx = Find(x)
    ry = Find(y)
    if rx == ry: return
    if rank[rx] < rank[ry]:
        parent[rx] = ry
    else if rank[rx] > rank[ry]:
        parent[ry] = rx
    else:
        parent[ry] = rx
        rank[rx] += 1

```

Ovo je već znatno bolje (jer se Union svodi na promenu jednog pokazivača), ali bez dodatnih trikova stablo može da postane izduženo (kao lista), pa Find može da bude $O(n)$ u najgorem slučaju. Zato uvodimo **dve klasične optimizacije**:

3.1 Union by rank (ili union by size)

Kod spajanja dva stabla, uvek **manje/plići** stablo prikačimo ispod korena **većeg/dubljeg**. Time sprečavamo da stablo neopravdano raste u dubinu. `rank[x]` je gornja granica visine

podstabla sa korenom x (ili se koristi `size[x]` – broj elemenata u podstablu).

3.2 Path compression (kompresija putanje)

Prilikom svakog poziva `Find(x)`, kada dođemo do korena, sve elemente na putu direktno “prikačimo” na koren:

```
Find(x):
    if parent[x] != x:
        parent[x] = Find(parent[x])    // rekurzivna kompresija
    return parent[x]
```

Ovo drastično spljoštava stablo za buduće upite, jer sledeći `Find` poziv nad bilo kojim elementom sa te putanje postaje skoro $O(1)$.

3.3 Vremenska složenost

Kada se kombinuju **union by rank** i **path compression**, niz od m operacija nad n elemenata izvršava se u vremenu:

$$O(m \cdot \alpha(n))$$

gde je $\alpha(n)$ **inverzna Ackermanova funkcija** – raste toliko sporo da je za sve praktične vrednosti n (čak i $n = 2^{65536}$) manja od 5. Zbog toga se u praksi ova struktura smatra **praktično konstantnom** po operaciji – $O(1)$ amortizovano.

3.4 Poređenje

	Naivna struktura (niz labela)	Union-Find (šuma + kompresija + rank)
MakeSet	$O(1)$	$O(1)$
Find	$O(1)$	$O(\alpha(n)) \approx O(1)$ amortizovano
Union	$O(n)$	$O(\alpha(n)) \approx O(1)$ amortizovano
m operacija ukupno	$O(n \cdot m)$	$O(m \cdot \alpha(n))$
Memorija	$O(n)$	$O(n)$

Ključna razlika: naivna struktura prebacuje sav “teret” na Union, dok union-find strukturu pravi jednako laganom za obe operacije korišćenjem stabla i dve heurističke optimizacije.

4. Implementacija u C++ (kompletan primer)

```
#include <bits/stdc++.h>
using namespace std;

class DisjointSet {
private:
    vector<int> parent, rank_;
    int components;

public:
    DisjointSet(int n) : parent(n), rank_(n, 0), components(n) {
```

```

        iota(parent.begin(), parent.end(), 0); // parent[i] = i
    }

    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]); // path compression
        return parent[x];
    }

    bool unite(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return false; // već su u istom skupu

        if (rank_[rx] < rank_[ry]) swap(rx, ry);
        parent[ry] = rx;
        if (rank_[rx] == rank_[ry]) rank_[rx]++;

        components--;
        return true;
    }

    bool sameSet(int x, int y) { return find(x) == find(y); }
    int numberOfSets() { return components; }
};

```

5. Primeri primene

5.1 Detekcija ciklusa / povezane komponente

```

int n = 6;
DisjointSet ds(n);
vector<pair<int,int>> edges = {{0,1},{1,2},{2,0},{3,4}};

for (auto [u, v] : edges) {
    if (!ds.unite(u, v))
        cout << "Grana (" << u << ", " << v << ") pravi ciklus!\n";
}
cout << "Broj povezanih komponenti: " << ds.numberOfSets() << "\n";
// Rezultat: grana (2,0) pravi ciklus, komponenti = 3 -> {0,1,2}, {3,4}, {5}

```

5.2 Kruskalov algoritam (MST)

```

struct Edge { int u, v, w; };

int kruskal(int n, vector<Edge> edges) {
    sort(edges.begin(), edges.end(),
        [](const Edge& a, const Edge& b) { return a.w < b.w; });

    DisjointSet ds(n);
    int mstWeight = 0, usedEdges = 0;

    for (auto& e : edges) {
        if (ds.unite(e.u, e.v)) { // grana ne pravi ciklus
            mstWeight += e.w;
            usedEdges++;
        }
    }
}

```

```

        if (usedEdges == n - 1) break;
    }
}
return mstWeight;
}

```

Ovde union-find garantuje da svaka provera “da li grana pravi ciklus” i njeno dodavanje košta skoro $O(1)$, pa je ukupna složenost Kruskalovog algoritma dominirana sortiranjem grana: **$O(E \log E)$** .

5.3 “Prijateljski krugovi” / equivalence klase

Zadatak: dat je spisak parova ljudi koji se poznaju; odrediti koliko postoji nezavisnih “krugova prijatelja”.

```

int friendCircles(int n, vector<pair<int,int>>& pairs) {
    DisjointSet ds(n);
    for (auto& [a, b] : pairs) ds.unite(a, b);
    return ds.numberOfSets();
}

```

6. Zaključak

- Naivna struktura (niz labela) je jednostavna, ali Union operacija je linearna po broju elemenata, što je neprihvatljivo za veliki broj operacija.
- Union-Find struktura (šuma sa **union by rank/size** i **path compression**) svodi obe ključne operacije na praktično konstantno amortizovano vreme $O(\alpha(n))$.
- Zbog jednostavnosti implementacije i izuzetne efikasnosti, union-find je standardni alat u algoritmima na grafovima (MST, povezanost, detekcija ciklusa) i u mnogim drugim oblastima gde je potrebno dinamički grupisati elemente u skupove.